

DS4440: Quiz 1

Name: _____

This quiz is closed-notes, closed-computer. Approximate numerical answers are fine. The goal is to check your understanding of logistic regression, gradient descent, validation loss, and (PyTorch) autograd.

Problem 1: SGD, manually (40 points)

Consider a binary logistic regression with no bias term. For one training example, suppose

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = 1, \quad w = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \alpha = 0.1.$$

The model prediction is

$$\hat{y} = \sigma(w^\top x), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

For logistic regression with binary cross-entropy loss, the gradient is

$$\nabla_w L = (\hat{y} - y)x.$$

(a) Compute the logit $z = w^\top x$.

$$z = w^\top x = 0 \cdot 1 + 0 \cdot 2 = 0.$$

(b) Compute the prediction $\hat{y}$.

$$\hat{y} = \sigma(0) = 0.5.$$

(c) Compute $\nabla_w L$.

$$\nabla_w L = (\hat{y} - y)x = (0.5 - 1) \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} -0.5 \\ -1 \end{bmatrix}.$$

(d) Perform one SGD update:

$$w \leftarrow w - \alpha \nabla_w L.$$

What is the new value of w ?

$$w_{\text{new}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -0.5 \\ -1 \end{bmatrix} = \begin{bmatrix} 0.05 \\ 0.1 \end{bmatrix}.$$

Problem 2: Gradient with respect to a hidden state (40 points)

Suppose a model computes a two-dimensional hidden state h , and then uses this hidden state to produce a scalar logit z :

$$h = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}, \quad z = v^\top h, \quad \hat{y} = \sigma(z).$$

For one training example, suppose

$$h = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \quad v = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \quad y = 1.$$

You may use

$$\sigma(-1) \approx 0.27.$$

For binary cross-entropy loss with a sigmoid output, recall

$$\frac{\partial L}{\partial z} = \hat{y} - y.$$

(a) Compute the logit $z = v^\top h$.

$$z = v^\top h = 1 \cdot 2 + 3 \cdot (-1) = 2 - 3 = -1.$$

(b) Compute the prediction $\hat{y}$.

$$\hat{y} = \sigma(z) = \sigma(-1) \approx 0.27.$$

(c) Compute $\frac{\partial L}{\partial z}$.

$$\frac{\partial L}{\partial z} = \hat{y} - y = 0.27 - 1 = -0.73.$$

(d) Compute the gradient with respect to the hidden state:

$$\nabla_h L = \frac{\partial L}{\partial h}.$$

Since

$$z = v^\top h,$$

we have

$$\frac{\partial z}{\partial h} = v.$$

Therefore,

$$\nabla_h L = \frac{\partial L}{\partial z} \frac{\partial z}{\partial h} = (-0.73) \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} -0.73 \\ -2.19 \end{bmatrix}.$$

Problem 3: PyTorch gradient gotcha (20 points)

Consider the following simplified PyTorch training loop:

```
w = torch.tensor([0.2, -0.4], requires_grad=True)

for i in range(len(X)):
    y_hat = torch.sigmoid(X[i] @ w)
    loss = -(y[i] * torch.log(y_hat) + (1-y[i]) * torch.log(1-y_hat))
    loss.backward()

    with torch.no_grad():
        w -= alpha * w.grad
```

- (a) What important step is missing from this training loop? What needs to be added?

The loop does not clear the gradient after the update. It is missing a call to `w.grad.zero_()`.

- (b) What happens if we run this loop over many training examples without fixing the issue?

PyTorch accumulates gradients by default. Without clearing the gradient, each update includes stale gradients from previous examples, so the update is not the intended per-example SGD update.